FINCANTIERI CHALLENGE · SOFTWARE DEFINED VESSEL · TEAM 6

ShipOS

A resource governor that decides what mission equipment the ship can carry and run, and explains every decision in numbers.

- Offline
- Deterministic
- Explainable
- No ML in the decision path

How the backend works, in 11 pictures

ShipOS sits between the ship and its mission equipment

The hull, generators and cooling are fixed. Mission equipment ("guests") is swapped per mission. Every guest competes for the same finite budgets.

FIXED

Ship platform

What the hull provides

- ⚡ 2 generators, 2 power lines
- ❄ Seawater cooling loop
- ▦ Deck bays, hangar, mast
- 📡 SATCOM, radio, acoustic links
- ▤ 4-node compute rack

status 1×/s → power, sea temp, crew

THE GOVERNOR

ShipOS

Checks every request against **8 budgets**, picks the least costly fix for the mission, and writes down why.

- **D1** Can it come aboard?
- **D2** Where does it go?
- **D3** Who goes first?
- **D4** What to cut back?

requests (cards) ← / → decisions

SWAPPABLE

Mission equipment

Each one ships with a data card

- **G1** Passive radio sensor
- **G2** Ocean & weather module
- **G3** Seabed inspection vehicle
- **G4** VTOL rescue drone
- **G5** Emergency comms node

👤 **Operator & crew see every decision with its reason, and confirm safety-critical steps**

In one line Air-traffic control for power, heat, space and time on board.

Source: docs/PLAN.md §0, §2

Eight finite budgets, all checked on every decision

HARD breaking it means **no** or **yes, if…** **ELASTIC** breaking it means **yes**, with a stated loss of capability

Power
HARD

Total and per line. Only **100 kW** for guests with one generator down.

Heat
HARD

Every kW used becomes 1 kW of heat. Cooling falls as the sea warms: **120 kW at 30 °C**.

Deck space
HARD

Bay 1 (whole or two halves), Bay 2, hangar, ramp. Bays can clash.

Mast
HARD

3 slots, 60 kg each. Scarce, and high weight counts triple for stability.

Mass & stability
HARD

10 t mission margin; **12 t** stability-weighted budget.

Data links
ELASTIC

SATCOM 2 Mbit/s at 70% availability; zero under radio silence.

Compute
MIXED

4 nodes, 80% allocatable. CPU share is elastic; accelerators are whole units (**3 usable**).

Response time
HARD

Admission ≤ 30 s · right-of-way ≤ 2 s · first cut ≤ 5 s · safety ≤ 200 ms.

Why it matters Power is not the only limit. In warm water, cooling can run out before power does.

Values: data/ship/reference_vessel.yaml (Annex A)

Four questions, one physics engine

All four decisions ask the same "does it fit?" check. They differ only in *what options they try* and *what they optimise*. The physics lives in one place, so the answers never contradict each other.

D1 · Admission

≤ 30 s

"Can this new equipment come aboard?" Tries removing, throttling or interlocking others. Answers **YES** **YES-IF** **NOT NOW** **NO**

D2 · Placement

part of D1

"Where should it go?" Tries every bay, slot and power line. Keeps the most headroom and future flexibility, and lists the top rejected layouts.

SHARED

Feasibility kernel

For a given loadout, check all 8 budgets in every worst-case condition and return the tightest one

`engine/kernel.py`

D3 · Right of way

≤ 2 s

"Two guests need conflicting ship states. Who wins?" Drops unsafe options, then maximises mission value. States the cost to the loser.

D4 · Degradation

≤ 5 s / 60 s

"A generator tripped. What do we cut?" Throttle before standby, standby before shutdown. Never cuts safety-critical equipment or the D3 winner.

D5 Crew policy gate: confirms or blocks actions by crew size (14 / 6 / 0)

D6 Physical Account: every number, its source and where it is used

Design choice Change one number on a card and every decision updates consistently.

Six steps from request to recorded answer

The same pipeline runs for every decision. Nothing is learned, guessed or scripted.

1

Read inputs

Ship facts, equipment cards, mission priorities, operating envelope. All plain YAML files.

2

Check every budget

All 8 budgets, in every worst case the mission allows (see next slide).

3

Find the binding limit

The single budget with the least headroom. This becomes the reason.

4

Try every fix

Remove, throttle, defer or interlock others. The space is small, so we try them all.

5

Rank by mission

Cheapest loss of mission value wins. Weights come from the swappable priority file.

6

Explain & record

A sentence built from the binding numbers, appended to a tamper-evident log.

Why try every option?

5 guests × up to 4 modes ≈ 1,000 combinations. That takes milliseconds, gives the same answer every time, and produces the ranked list of alternatives the brief asks for.

Why "binding limit"?

Judges and crew don't need 40 numbers. They need the one that decides, with who is using it.

How does it scale?

A built-in guard switches to a greedy search if the space grows. A constraint solver (OR-Tools) is the next step.

Result Every answer comes with its reason, its alternatives and the cost to whoever loses.

A "yes" must hold in every corner of the mission envelope

We don't check today's weather. We check the worst conditions the mission allows, so the ship never has to unload equipment mid-mission.

	Slow speed	Cruise speed
2 generators	✓ fits 30 °C sea	✓ fits 30 °C sea
1 generator down	✗ 101.5 / 100 kW binding corner (reported)	✗ 101.5 / 100 kW also short

Contingency (dashed): cooling loop degraded at 30 °C. Not in the default envelope, so each verdict pre-computes a cut-back plan for it instead.

Example: the G4 rescue drone

1. Both two-generator corners pass easily (200 kW available).
2. In both one-generator corners, guests may draw only **100 kW**. Drone charging plus the G2 winch would need **101.5 kW**.
3. So the verdict is **YES-IF** : "the winch waits while the drone charges, *only when a generator is down*."

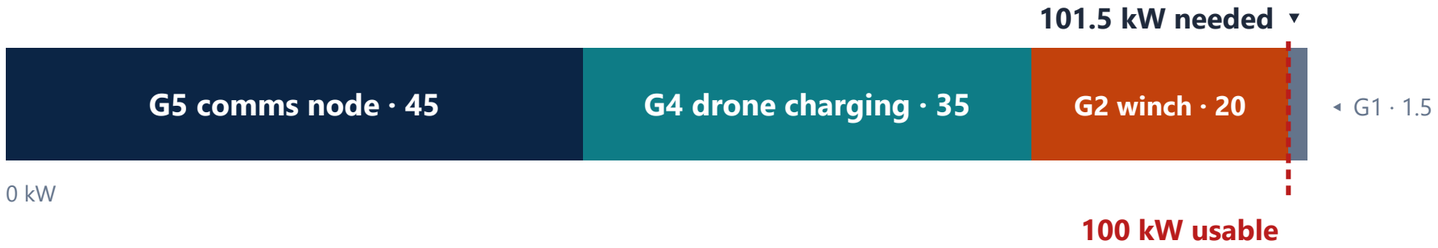
→ The jury can widen the envelope (e.g. include a degraded loop) in one YAML line, and the verdict recomputes.

Principle Admit only what works in the worst case. Then say exactly what changes in that case.

Jury hands us the G4 drone card. What does ShipOS say?

Scenario A: G1, G2 and G5 already installed. Output below comes from running `engine.admission.decide()`, not from a slide.

Power demand in the binding corner (slow, 30 °C, one generator down)



YES-IF **Fix:** interlock G2's winch while G4 charges, **only when a generator is down**.

RATIONALE, GENERATED BY THE ENGINE

"Admit G4 if G2's winch waits while G4 charging with one genset down: at slow speed in 30 °C water with one genset down, guests need **101.5 kW** against **100 kW** usable (1.5 kW short; G5 45, G4 35, G2 20, G1 1.5 kW); this costs 0.05 capability against 0.2 returned."

What else the decision carries

📍 **Placement:** G4 → hangar

📋 **Ranked alternatives:** throttle or remove others, each with its cost

⚡ **Holds with one generator down?** Only with the interlock

❄ **Contingency:** a cut-back list if the cooling loop degrades at 30 °C

Jury changes one number

Set G4's mass to 20 t and the verdict flips to **NO**, binding on hangar mass. Same code, new reason.

Card G3 instead

YES-IF displace G5. Binding: Bay 1 holds 2 units, G3 + G5 need 3.

tests/engine/test_admission.py · output captured from merged main

Same ship, same physics, different mission → different answer

Swap one file (`priorities/survey_primary.yaml` ↔ `sar_primary.yaml`) and the ranking changes. Both answers are computed, neither is scripted.

	PRIORITY FILE **Survey primary** Seabed inspection matters most	PRIORITY FILE **Search & rescue primary** Rescue drone and comms matter most
D1 Jury card G3 (Bay 1 is full)	**YES-IF** Displace G5. Costs 0.2 capability, returns 1.0. Worth it for a survey.	**NO** Cheapest fix costs 0.5 capability, more than the 0.2 G3 returns on a rescue mission.
D4 Generator down, two batteries charging 106.5 kW > 100 kW	**THROTTLE** G5 comms node to low power: **45** → **8 kW** (37 kW recovered). Survey keeps running.	**DEFER** G3's charge waits **5 min** until the drone finishes. Comms node kept at full power for the rescue.

What's in the file: a weight per guest, an asset-loss penalty, "protect the D3 winner", tie-breaks.

What's not: any rule. "Mission-critical" is derived from the weights, never hard-coded.

For the jury Hand us a new priority file and the governor re-ranks every decision without a code change.

tests/engine/test_admission.py · test_shedding.py (D4 on a labelled controlled input)

8 · A LIVE RUN

Scenario B, second by second

A seeded simulator ticks once per second. Events trigger the real engine. Loadout: G1, G3, G4, G5 · 24 °C sea · 3 knots.

`t = 0 s`

Steady state

Survey under way, drone flying. All budgets green.

`t = 480 s · battery report`

D3 right-of-way

Drone must land; G3 needs quiet. **Drone recovery wins.** G3 paused (survey) or suspended (SAR).

`t = 720 s · generator trip`

D4 degradation

Draw is 49.5 kW vs 100 kW. Answer: **NO ACTION** . A trip is not an overload.

`t = 840 s · prep ready`

D5 crew gate

Ship is ready, but the drone **keeps flying** until a human confirms the deck is clear.

`t = 841 s · confirmed`

Drone recovers

Operator confirmation recorded; recovery starts on the next tick.

👥 14 crew: human confirms, drone lands

👥 6 crew: deck clearing starts early from the battery forecast

👥 6 or 0 crew, deck occupied (baseline B): denied or blocked, no landing

Honest by design Human confirmation is an input we wait for, never something the planner assumes.

data/scenarios/B.timeline.yaml · tests/api/test_execution_acceptance.py

Five layers, all running on one laptop, fully offline

Python 3.11 backend, a plain browser front end, no cloud services, no build step, no external solver.

5 · Interface	Browser dashboard: ship view (3D, offline), 8 gauges, decision log, card editor, scenario controls	`vanilla JS · SVG · three.js (vendored)`
4 · API	Sessions, commands, live push over WebSocket, evidence export, replay download	`FastAPI · uvicorn · WebSocket`
3 · Simulator	Seeded 1 Hz clock, event timeline, consumption profiles, weather from cache, execution controller, crew policy (D5)	`shipos/sim · shipos/crew`
2 · Engine	**Pure functions.** Physics for 8 budgets, feasibility kernel, D1–D4, rationale renderer, ship state machine	`shipos/physics · shipos/engine`
1 · Data	Ship, guest cards, priorities, envelope, scenarios, assumption registry, all validated against frozen contracts	`YAML · pydantic · JSON Schema`

Decision log

Append-only JSONL. Each line carries the hash of the one before, so any edit breaks the chain.

Replay

The exported bundle (inputs + seed + log) re-runs the engine and must reproduce the same final hash.

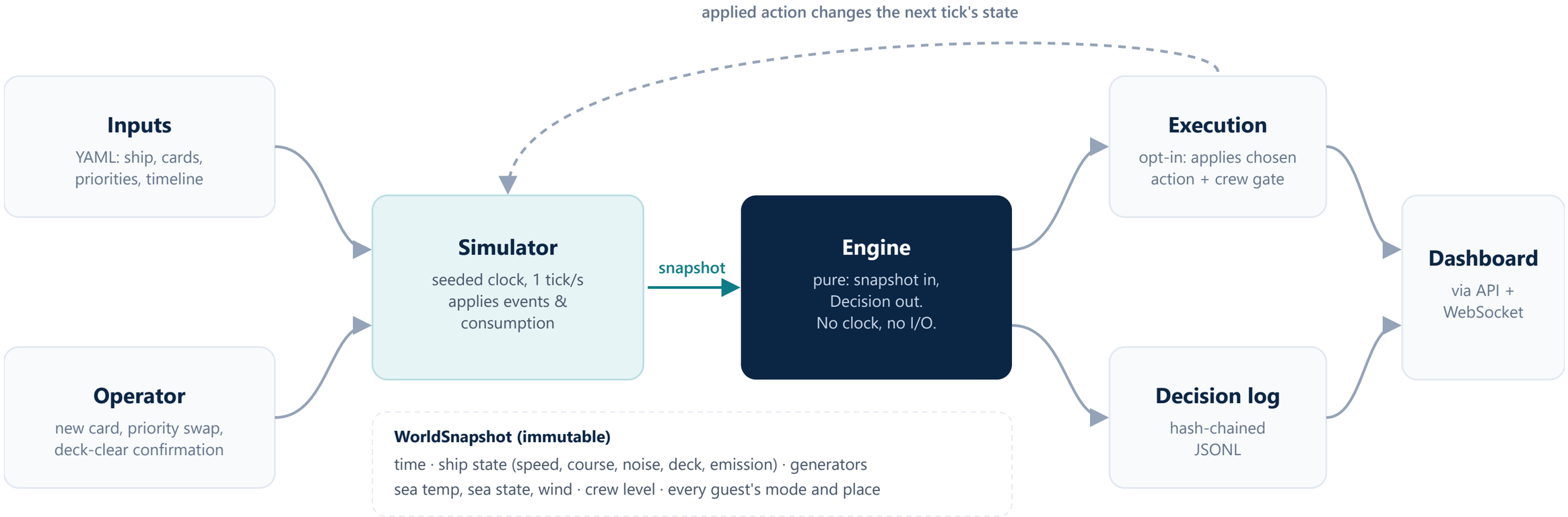
Engineering

uv for locked dependencies, pytest, ruff, GitHub Actions CI with guardrail checks.

pyproject.toml · shipos/ · shipos/api/static/

One tick of the system

Every second the simulator builds an immutable snapshot of the world. If an event needs a decision, the pure engine answers and the log records it.



Why pure? Same inputs and seed always give the same decisions, so any run can be replayed and audited.

shipos/sim/runner.py · shipos/contracts.py WorldSnapshot · shipos/sim/log.py

Six guarantees, each enforced by an automated check

These aren't promises in a README. CI blocks any change that breaks them.

1 No magic numbers

Engine code may not contain numbers. Each one lives in a registry with its **value, unit, range, reason and source**.

`✓ tests/test_no_literals.py`

2 No canned explanations

Every rationale is assembled from the binding constraint. A test fails if a sentence lacks its numbers.

`✓ tests/engine/test_rationale.py`

3 Deterministic

No randomness or wall clock inside the engine. Same seed and timeline → byte-identical log hash.

`✓ tests/sim/test_runner.py`

4 Checked against the brief

Expected results are computed from Annex inputs inside the tests, never copied from engine output.

`✓ tests/physics · tests/engine`

5 Tamper-evident

The decision log is hash-chained. Replay re-runs the decisions, not just the hashes.

`✓ shipos/sim/log.py verify_jsonl`

6 Fails closed

Bad or ambiguous input is rejected as a whole. Unsupported actions stay blocked rather than guessed.

`✓ tests/api/test_import_card.py`

🔍 **We checked the brief's arithmetic: "any four guests exceed 10 t" is false ({G1,G3,G4,G5} = 9.61 t)**

🛡 **Civil use only · no ML in decisions**

Bottom line Every decision can be traced to a number, every number to a source.